

# Relatório Técnico Consolidado — Ecossistema Feel / FeelWorks

**Objetivo deste documento:** fornecer a outra IA um contexto suficientemente preciso para avaliar plataformas **Open Source / Low-Code / AI-Agentic disponíveis no GitHub** e determinar qual delas oferece a melhor base para construir o produto FeelWorks.

**Importante:** este documento separa deliberadamente o que já foi definido, o que foi explorado como hipótese e o que ainda precisa ser decidido. Não deve ser interpretado como uma especificação técnica fechada.

## 1. VISÃO GERAL E ESCOPO DO PROJETO

### 1.1. O que está sendo construído

A Feel está evoluindo de uma operação tradicional de serviços criativos para um **ecossistema de inteligência, tecnologia, criatividade e pessoas**.

A tese central é:

**A Feel recebe problemas de negócio, entende o contexto, identifica as capacidades necessárias e combina dados, inteligência, tecnologia, criatividade e pessoas para construir soluções.**

O software é uma infraestrutura para viabilizar esse modelo.

Portanto, **não se trata simplesmente de construir um SaaS de gestão**, um marketplace de freelancers ou um gerador de conteúdo com IA.

O sistema precisa eventualmente funcionar como uma camada de **inteligência + orquestração + operação**.

### 1.2. Arquitetura conceitual do ecossistema

O ecossistema está sendo dividido em diferentes braços:

#### FeelMind — Inteligência

Camada responsável por:

- dados;
- análise;
- inteligência;
- IA;
- estratégia;

- recomendações;
- memória organizacional;
- Creative Intelligence;
- interpretação de contexto.

A ideia conceitual é:

***FeelMind = o que devemos fazer?***

---

## **FeelWorks — Plataforma**

É a camada tecnológica/operacional onde as atividades efetivamente acontecem.

Responsável por conectar:

- clientes;
- projetos;
- inteligência;
- criação;
- profissionais;
- parceiros;
- assets;
- aprovações;
- resultados.

Conceitualmente:

***FeelWorks = onde o ecossistema trabalha.***

---

## **FeelMakers — Rede de talentos**

Rede de:

- designers;
- criadores;
- freelancers;
- profissionais liberais;
- consultores;
- especialistas;
- produtoras;
- outros talentos da indústria criativa.

O objetivo futuro é permitir descoberta, curadoria, matching e contratação baseada em **capacidades e evidências**, e não simplesmente em currículo ou portfólio.

## Feel Partners — Rede de parceiros

Conceito em desenvolvimento.

Pode incluir:

- agências;
- empresas especializadas;
- consultorias;
- fornecedores;
- especialistas;
- parceiros tecnológicos.

A ideia é que cada parceiro seja reconhecido por **capacidades específicas**, podendo ter:

- certificações;
- histórico;
- reputação;
- cases;
- competências;
- resultados;
- disponibilidade;
- relacionamento com a Feel.

O sistema poderá utilizar essas informações para recomendar parceiros adequados a determinado problema.

---

## FeelSpace — Camada física

Abrange:

- estúdios;
- espaços de criação;
- experiências;
- workshops;
- ativações;
- ambientes físicos;
- operações presenciais.

É uma extensão física do ecossistema, não necessariamente uma funcionalidade central do primeiro MVP.

---

## 1.3. Quem são os usuários?

A resposta é: **não é um sistema estritamente interno.**

A visão de longo prazo é claramente **multiusuário e multi-tenant**, com experiências diferentes para diferentes tipos de participantes.

## Perfil A — Cliente / Empresa

Pode ser:

- empresa;
- marca;
- instituição;
- organização;
- cliente político/institucional;
- eventualmente outros tipos de organização.

Necessidades:

- abrir demandas;
- acompanhar projetos;
- acessar inteligência;
- visualizar recomendações;
- fornecer informações;
- aprovar materiais;
- acessar Brand Center;
- acessar assets;
- acompanhar resultados;
- interagir com equipes e parceiros.

---

## Perfil B — Equipe interna Feel

Usuários responsáveis por:

- estratégia;
- relacionamento;
- gestão de projetos;
- direção criativa;
- inteligência;
- curadoria;
- operação;
- gestão dos parceiros.

Terão permissões mais amplas.

## Perfil C — Profissional / Creator

Usuários da FeelMakers.

Podem:

- criar perfil;
- cadastrar competências;
- apresentar portfólio;
- receber oportunidades;
- participar de projetos;
- construir reputação;
- obter certificações;
- utilizar ferramentas da plataforma.

## Perfil D — Agência / Partner

Agências e empresas especializadas.

Possuem perfil organizacional, potencialmente com:

- equipe;
- competências;
- cases;
- certificações;
- histórico;
- projetos;
- oportunidades;
- reputação;
- indicadores.

## Perfil E — Administrador

Responsável pela governança:

- usuários;
- organizações;
- permissões;
- projetos;
- integrações;
- billing;

- parceiros;
- configurações;
- auditoria;
- dados.

---

## 1.4. Estratégia de evolução do produto

Um ponto importante já discutido:

**não existe intenção de construir toda essa visão de uma vez.**

A orientação estratégica é:

### Fase 1

Construir ferramentas internas / AI-enabled services para resolver gargalos reais da operação Feel.

### Fase 2

Transformar componentes validados em produtos/serviços tecnológicos comercializáveis.

### Fase 3

Evoluir para uma plataforma mais ampla, com clientes externos, rede de profissionais e parceiros.

### Fase 4

Possível modelo B2B2C/ecossistema.

Isso é importante para a escolha da plataforma: a solução ideal deve permitir **começar pequeno e evoluir**, sem exigir que toda a arquitetura futura seja construída desde o primeiro dia.

---

## 2. ARQUITETURA DE DADOS E BACKEND

### 2.1. Complexidade esperada

A aplicação tende a exigir **modelagem relacional relativamente complexa**.

Não é apenas:

Usuário → Dados

Existe uma rede de relações:

Organization  
↓  
Users

↓  
Projects  
↓  
Briefings  
↓  
Tasks  
↓  
Assets  
↓  
Approvals  
↓  
Deliverables  
↓  
Results

Ao mesmo tempo:

Projects  
↓  
Required Capabilities  
↓  
Professionals / Agencies  
↓  
Partners  
↓  
Skills / Certifications  
↓  
Reputation / Performance

E:

Organization  
↓  
Brand  
↓  
Brand Memory  
↓  
Decisions  
↓  
Assets  
↓  
Campaigns  
↓  
Performance

Portanto, **um banco relacional é provavelmente mais adequado ao core transacional do sistema do que uma arquitetura puramente documental/noSQL.**

## 2.2. Entidades que provavelmente existirão

A modelagem final ainda não foi fechada, mas o sistema provavelmente terá entidades como:

### Identity / Access

- User
- Organization
- Role
- Permission
- Membership
- Team

### Client / Business

- Organization
- Brand
- Business Unit
- Contact

### Projects

- Project
- Brief
- Scope
- Task
- Milestone
- Deliverable
- Comment
- Approval

### Creative

- Asset
- Asset Version
- Campaign
- Content
- Creative Direction
- Brand Guideline
- Template

### Intelligence

- Insight

- Recommendation
- Analysis
- Data Source
- Decision
- Strategic Objective
- Memory
- Knowledge Item

## Network

- Professional
- Agency
- Partner
- Capability
- Skill
- Certification
- Case
- Availability
- Reputation Metric

## Commercial

- Proposal
- Contract
- Subscription
- Invoice
- Payment
- Revenue Share

## AI

- Agent
- Agent Task
- Conversation
- Prompt/Instruction
- Tool
- Knowledge Base
- Retrieval Source
- Execution
- Human Approval

## 2.3. Banco de dados considerado

Não existe, no histórico consolidado, uma decisão definitiva de banco de dados.

**PostgreSQL é uma forte candidata e parece particularmente adequada ao modelo relacional previsto**, mas isso ainda deve ser tratado como decisão arquitetural a validar.

Não há decisão consolidada de utilização de:

- MongoDB;
- MySQL;
- Supabase;
- Firebase;
- outro banco específico.

Portanto, a ferramenta de IA que avaliar plataformas deve **não assumir PostgreSQL como decisão fechada**, mas considerar como requisito desejável:

*suporte robusto a PostgreSQL ou banco relacional equivalente.*

---

## 2.4. Arquitetura potencial

Já foi discutida anteriormente a ideia de uma arquitetura **modular/hexagonal**, justamente para evitar acoplamento excessivo entre:

- domínio;
- banco;
- integrações;
- IA;
- interface;
- serviços externos.

Isso é particularmente importante porque o produto deverá eventualmente integrar múltiplos serviços de IA, APIs e provedores.

A plataforma escolhida não deve dificultar uma eventual migração para código próprio.

---

## 3. WORKFLOWS E INTEGRAÇÕES

### 3.1. Integrações já consideradas conceitualmente

O sistema deverá potencialmente integrar:

#### Redes sociais

Especialmente:

- Instagram;
- TikTok;
- outras plataformas sociais relevantes.

Objetivos:

- métricas;
  - publicação;
  - conteúdo;
  - calendário;
  - análise;
  - monitoramento.
- 

## IA

Já foram considerados:

- Anthropic / Claude;
- Claude Code;
- modelos generativos;
- frameworks multi-agent;
- ferramentas externas;
- eventualmente MCP.

A escolha do fornecedor de modelo **não deve ser hard-coded na arquitetura.**

Idealmente deve existir uma camada de abstração de LLM.

---

## Cloud

Já foi considerada integração com serviços cloud.

A infraestrutura específica ainda não está fechada.

---

## GitHub

GitHub faz parte do processo de desenvolvimento.

Também é relevante para a eventual escolha da plataforma Open Source.

---

## Ferramentas de organização

Foram utilizados/considerados:

- Notion;
- Obsidian;

- GitHub;
- Vercel.

Esses itens não devem ser automaticamente considerados integrações obrigatórias do produto final.

---

## 3.2. Workflows principais

### Workflow 1 — Novo problema / briefing

```
Cliente
↓
Novo projeto
↓
Briefing
↓
Coleta de contexto
↓
FeelMind
↓
Análise
↓
Recomendação
```

### Workflow 2 — Recomendação de capacidades

```
Problema
↓
Análise
↓
Identificação de capacidades necessárias
↓
Busca na rede
↓
Matching
↓
Ranking
↓
Profissionais / Agências recomendados
↓
Humano valida
```

### Workflow 3 — Criação

Dados  
↓  
Contexto  
↓  
Estratégia  
↓  
Creative Intelligence  
↓  
Direcionamento criativo  
↓  
IA / Designer  
↓  
Asset  
↓  
Revisão  
↓  
Cliente  
↓  
Aprovação

## Workflow 4 — Aprovação

Deve existir human-in-the-loop.

Exemplo:

IA gera recomendação  
↓  
Humano revisa  
↓  
Aprova / altera / rejeita  
↓  
Execução

O sistema **não deve assumir autonomia irrestrita da IA em decisões críticas.**

## Workflow 5 — Feedback / aprendizado

Projeto  
↓  
Execução  
↓  
Resultado  
↓  
Feedback  
↓  
Memória



FeelMind



Próximas recomendações

Essa camada é central para a visão de longo prazo.

## 3.3. Eventos

A arquitetura deverá potencialmente suportar:

- webhooks;
- eventos de alteração de status;
- uploads;
- aprovação;
- conclusão de tarefas;
- entrada de dados;
- atualização de métricas;
- execução de agentes;
- notificações.

Também existe potencial para:

- cron jobs;
- tarefas periódicas;
- sincronização de APIs;
- atualização de dashboards.

## 4. CAPACIDADES DE IA E SISTEMAS AGÊNTICOS

Esta é uma das partes mais importantes do sistema.

### 4.1. IA não deve ser apenas um chatbot

A visão não é simplesmente:

"Adicionar um ChatGPT dentro do software."

A IA deve funcionar como uma **camada operacional e cognitiva do ecossistema**.

### 4.2. Casos de uso de IA

## Inteligência de negócio

- analisar dados;
  - identificar padrões;
  - gerar insights;
  - identificar oportunidades;
  - detectar riscos;
  - interpretar tendências;
  - comparar concorrentes.
- 

## Estratégia

- transformar dados em recomendações;
  - sugerir ações;
  - priorizar oportunidades;
  - sugerir campanhas;
  - estruturar planos.
- 

## Creative Intelligence

A IA deverá receber:

- contexto da marca;
- dados;
- posicionamento;
- objetivos;
- histórico;
- assets;
- decisões anteriores.

E produzir direcionamentos para:

- cor;
- tipografia;
- composição;
- linguagem visual;
- tom;
- formatos;
- aplicações;
- estratégia de conteúdo.

O objetivo não é apenas "gerar uma imagem bonita".

É:

***gerar uma solução criativa coerente com a inteligência acumulada sobre aquela marca.***

---

## 4.3. Memória da marca

Um dos conceitos mais importantes.

Cada organização deve possuir uma espécie de **Memória Viva da Marca**.

Ela poderá armazenar:

- documentos;
- identidade;
- guidelines;
- decisões;
- campanhas;
- assets;
- histórico;
- aprovações;
- resultados;
- aprendizados;
- contexto.

A IA utiliza esse contexto nas interações futuras.

---

## 4.4. RAG

**RAG é um requisito fortemente desejável.**

A IA deverá conseguir recuperar informações de:

- documentos;
- Brand Center;
- assets;
- histórico de projetos;
- decisões;
- conhecimento interno;
- dados estruturados.

O sistema idealmente terá:

Structured Data

+

Documents

+

Vector Knowledge

+

Conversation Context

+

Business Rules

## 4.5. Tool use

Os agentes deverão potencialmente utilizar ferramentas externas.

Exemplos:

- APIs sociais;
- analytics;
- bancos de dados;
- ferramentas de criação;
- serviços de IA;
- automações;
- sistemas externos.

A arquitetura deve favorecer **tool calling** e, potencialmente, MCP.

## 4.6. Multi-agent

Existe interesse em uma arquitetura multi-agent.

Não necessariamente vários agentes conversando livremente.

Uma arquitetura mais provável seria:

Orchestrator

↓



Data Strategy Creative Operations

Agent Agent Agent Agent

Cada agente possuiria:

- objetivo;
- contexto;

- ferramentas;
- permissões;
- memória;
- outputs estruturados.

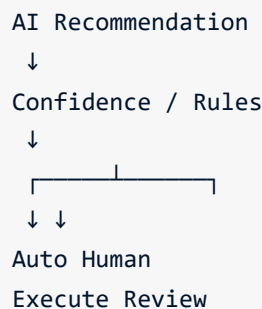
## 4.7. Human-Agent Handoff

Isso é essencial.

A IA deve saber quando:

- pode executar;
- precisa solicitar aprovação;
- precisa solicitar informação;
- deve escalar para humano;
- não possui confiança suficiente.

Exemplo:



## 4.8. Agentes não devem possuir autoridade ilimitada

Decisões como:

- publicação;
- contratação;
- aprovação financeira;
- alteração de estratégia;
- alteração de identidade;
- comunicação política;
- ações sensíveis;

devem possuir mecanismos de aprovação e controle.

Isso também será especialmente importante em clientes políticos.

---

## 5. UI / UX

### 5.1. Complexidade

O frontend será **consideravelmente mais complexo do que um dashboard CRUD convencional**.

Não basta um construtor drag-and-drop.

Existe necessidade de interfaces customizadas para diferentes jornadas.

---

### 5.2. Princípio principal

A UI deve ser **role-based e context-aware**.

A mesma plataforma muda de acordo com quem está utilizando.

#### Cliente

Vê:

- projetos;
- aprovações;
- resultados;
- inteligência;
- marca;
- assets.

#### Profissional

Vê:

- perfil;
- capacidades;
- oportunidades;
- projetos;
- reputação;
- ferramentas.

#### Agência

Vê:

- organização;
- equipe;

- cases;
- oportunidades;
- projetos;
- certificações.

## Equipe Feel

Vê:

- operação;
- projetos;
- inteligência;
- rede;
- distribuição;
- clientes.

---

## 5.3. Navegação conceitual

A navegação anteriormente imaginada possui grandes áreas:

- **Home**
- **Projetos**
- **Inteligência**
- **Marcas**
- **Pessoas**
- **Rede**
- **Oportunidades**
- **Conteúdos**
- **Financeiro**
- **Configurações**

Isso é uma arquitetura conceitual, não uma especificação definitiva.

---

## 5.4. Brand Center

Cada cliente/organização deverá possuir um espaço próprio.

Pode conter:

```
Brand
├─ Identity
├─ Guidelines
```

- Assets
- Campaigns
- Content
- Decisions
- Approvals
- History
- Knowledge

## 5.5. Gestão de assets

O sistema deverá potencialmente funcionar como um pequeno DAM:

- upload;
- versionamento;
- organização;
- metadados;
- permissões;
- aprovação;
- histórico;
- busca;
- recuperação por IA.

## 5.6. UI de IA

A IA não deve estar limitada a uma tela "chat".

Idealmente deve existir uma combinação de:

- chat contextual;
- recomendações inline;
- ações sugeridas;
- copilots;
- automações;
- insights;
- painéis;
- agentes especializados.

Exemplo:

Dentro de um projeto:

***FeelMind identificou 3 oportunidades.***

O usuário pode:

- visualizar;
  - perguntar por quê;
  - aceitar;
  - editar;
  - executar.
- 

## 5.7. Nível de customização

A plataforma precisa permitir **frontend bastante customizável**.

O sistema não é adequado a uma solução que apenas gere automaticamente CRUDs e dashboards genéricos.

Idealmente a plataforma escolhida deve permitir:

- React ou equivalente;
  - componentes customizados;
  - APIs;
  - lógica própria;
  - design system;
  - responsive;
  - interfaces específicas por role.
- 

## 6. REQUISITOS NÃO FUNCIONAIS E INFRAESTRUTURA

### 6.1. Cloud / Self-hosting

Ainda não existe decisão final.

Entretanto, por se tratar de:

- dados de clientes;
- informações estratégicas;
- dados de marketing;
- assets;
- informações potencialmente sensíveis;
- eventualmente dados políticos;

é altamente desejável que a plataforma avaliada suporte **self-hosting**.

Preferência arquitetural:

Docker

+

Cloud VPS / Cloud Provider

com possibilidade futura de infraestrutura mais sofisticada.

Kubernetes não foi estabelecido como requisito atual.

---

## 6.2. Open Source

Este é um requisito explícito da pesquisa atual.

A ferramenta que será escolhida deve ser realmente útil como base tecnológica de longo prazo.

Importante verificar:

- licença;
- possibilidade de uso comercial;
- capacidade de fork;
- dependências proprietárias;
- lock-in;
- comunidade;
- atividade do GitHub;
- frequência de releases;
- documentação;
- número e qualidade dos contributores.

---

## 6.3. Segurança

A plataforma deve suportar, idealmente:

### Authentication

- email/password;
- OAuth;
- eventualmente SSO.

### Authorization

RBAC é desejável.

Possivelmente:

```
Platform Admin
Organization Admin
Project Manager
Strategist
Creative
Partner
Professional
Client
Viewer
```

Mas o modelo definitivo ainda não foi fechado.

## 6.4. Multi-tenancy

**Muito importante para a visão futura.**

A plataforma deverá eventualmente separar dados entre organizações.

Exemplo:

```
Feel
|
|— Organization A
|   |— Users
|   |— Brands
|   |— Projects
|
|— Organization B
|   |— Users
|   |— Brands
|   |— Projects
|
|— Organization C
```

O isolamento de dados deve ser uma preocupação arquitetural desde o início, mesmo que o MVP inicial seja interno.

## 6.5. LGPD / privacidade

Como a operação será brasileira e poderá lidar com dados pessoais, a arquitetura deve considerar:

- LGPD;
- consentimento quando aplicável;
- controle de acesso;
- logs;
- retenção;

- exclusão;
- exportação;
- minimização de dados;
- segregação entre organizações.

Não foi definida uma certificação formal específica.

## 6.6. Observabilidade

Ainda não especificado, mas uma plataforma agentic exigirá futuramente:

- logs;
- tracing;
- histórico de execução de agentes;
- custo por execução;
- latência;
- erros;
- auditoria;
- métricas de uso.

Especialmente importante para IA:

```
graph TD; Agent --> Prompt; Prompt --> Model; Model --> Tools; Tools --> Output; Output --> HumanApproval[Human Approval];
```

Toda essa cadeia deve ser observável.

## 7. PERFIL DA EQUIPE DE DESENVOLVIMENTO

### 7.1. Perfil atual

A iniciativa está sendo desenvolvida por uma equipe pequena, com forte interesse em:

- IA;

- automação;
- software;
- ferramentas agentic;
- desenvolvimento assistido por IA.

Não se trata de uma grande equipe tradicional de engenharia.

---

## 7.2. AI-assisted development

O desenvolvimento deverá aproveitar intensivamente ferramentas como:

- Claude;
- Claude Code;
- agentes de código;
- GitHub;
- plataformas AI-assisted.

Já houve experimentação com Claude Code.

A intenção é utilizar IA para acelerar significativamente o desenvolvimento.

---

## 7.3. Nível técnico desejável da plataforma

A plataforma escolhida não deve exigir que toda implementação seja feita manualmente desde o zero.

Mas também não pode prender o projeto em um builder fechado.

O cenário ideal é:

Low-Code  
+  
AI Agentic Development  
+  
Custom Code  
+  
Open Source

Ou seja:

*velocidade de prototipação sem sacrificar controle arquitetural.*

---

## 7.4. Característica desejada da ferramenta

Idealmente:

## Deve facilitar

- geração de CRUD;
- autenticação;
- database schema;
- APIs;
- workflows;
- integrações;
- agentes;
- RAG;
- dashboards;
- UI inicial.

## Mas deve permitir

- código customizado;
- APIs próprias;
- componentes React;
- customização de backend;
- acesso ao banco;
- integração com LLMs;
- webhooks;
- jobs;
- Docker;
- self-hosting.

---

# 8. O QUE É MVP E O QUE É VISÃO FUTURA

Essa distinção é **fundamental para avaliar as plataformas**.

## MVP mais próximo

O MVP já discutido anteriormente contempla principalmente:

### Conexão de contas

Conectar redes sociais e outras fontes.

### Métricas

Coletar e visualizar indicadores.

### Calendário

Planejamento de conteúdo.

Ideias de conteúdo

Sugestões assistidas por IA.

Recomendações de monetização

Sugestões baseadas em dados.

IA

Assistência para interpretação e tomada de decisão.

9. VISÃO DE PRODUTO MAIS AMPLA

Depois da validação, o sistema pode evoluir para:



10. MODELO DE NEGÓCIO QUE IMPACTA A ARQUITETURA

A visão comercial discutida é potencialmente **B2B2C**.

B2B

Empresas compram:

- inteligência;
- tecnologia;
- estratégia;
- soluções;
- execução;
- acesso à rede.

Possibilidade de tickets elevados.

---

## B2C / Professionals

Em uma fase posterior:

- creators;
- freelancers;
- profissionais liberais;
- indústria criativa.

Podem pagar assinaturas premium.

A plataforma poderia funcionar como uma espécie de **Professional/Career OS**, oferecendo:

- ferramentas;
  - IA;
  - oportunidades;
  - reputação;
  - networking;
  - capacitação;
  - acesso à rede.
- 

## Marketplace / Network

Possível monetização através de:

- assinaturas;
- comissões;
- projetos;
- revenue share;
- certificações;
- serviços premium.

Isso ainda é hipótese de modelo de negócio, não requisito imediato do MVP.

---

# 11. REQUISITOS ESPECÍFICOS PARA ESCOLHER A PLATAFORMA OPEN SOURCE

Ao avaliar projetos do GitHub, a outra IA deve pontuar fortemente os seguintes critérios.

## Obrigatórios / muito importantes

### 1. Open Source real

Verificar licença.

### 2. Self-hosting

Preferencialmente Docker.

### 3. PostgreSQL ou banco relacional robusto

### 4. API-first

### 5. Custom code

### 6. Integração com LLMs

### 7. Agent workflows

### 8. RAG

### 9. Webhooks

### 10. Authentication / RBAC

### 11. Multi-tenancy ou possibilidade razoável de implementação

### 12. Frontend altamente customizável

### 13. GitHub ativo

### 14. Comunidade saudável

### 15. Baixo vendor lock-in

## 12. CRITÉRIOS DE AVALIAÇÃO RECOMENDADOS

Sugestão de matriz:

| Critério                    | Peso |
|-----------------------------|------|
| Arquitetura extensível      | 15%  |
| AI / Agentic capabilities   | 15%  |
| Open Source / licença       | 10%  |
| Customização frontend       | 10%  |
| Backend / APIs              | 10%  |
| Database                    | 10%  |
| Self-hosting                | 8%   |
| Workflows / automação       | 8%   |
| RAG / Knowledge             | 5%   |
| Auth / RBAC / multi-tenancy | 5%   |
| Comunidade / GitHub         | 2%   |
| Documentação                | 2%   |

A ferramenta não deve ser escolhida simplesmente porque "gera apps com IA".

O principal critério é:

***Ela consegue ser a fundação de um produto real e complexo sem impedir a Feel de evoluir para código e arquitetura próprios?***

# 13. PRINCIPAIS RISCOS ARQUITETURAIS

## Risco 1 — Construir tudo cedo demais

A visão final é muito maior que o MVP.

Não transformar a primeira versão em:

- CRM;
- marketplace;
- DAM;
- BI;
- ERP;
- social media manager;
- agent platform;

- project management;
- career platform;

ao mesmo tempo.

---

## Risco 2 — Lock-in da ferramenta Low-Code

Uma ferramenta que acelera muito o início mas impede acesso ao código, banco ou APIs pode se tornar um problema.

---

## Risco 3 — IA como feature superficial

Adicionar chatbot não transforma o produto em AI-native.

A IA precisa estar incorporada aos workflows.

---

## Risco 4 — Modelo de dados insuficiente

Um builder que funciona bem para CRUD simples pode não suportar adequadamente:

- multi-tenancy;
  - projetos;
  - rede;
  - capacidades;
  - reputação;
  - memória;
  - agentes;
  - permissões.
- 

## Risco 5 — Confundir marketplace com rede de capacidades

A visão não é simplesmente listar freelancers.

A plataforma deverá eventualmente entender:

***qual capacidade é necessária para determinado problema e quem possui evidências de que consegue fornecê-la.***

---

# 14. DECISÕES AINDA NÃO FECHADAS

A outra IA **não deve assumir como decididos** os seguintes pontos:

- banco de dados definitivo;
- provedor cloud definitivo;

- arquitetura final de microservices vs modular monolith;
- framework definitivo de frontend;
- framework definitivo de agentes;
- modelo LLM principal;
- uso efetivo de MCP;
- sistema de billing;
- autenticação/SSO definitivo;
- modelo final de multi-tenancy;
- vector database;
- storage de assets;
- infraestrutura Kubernetes;
- sistema definitivo de observabilidade;
- modelo final de marketplace;
- modelo final de assinatura;
- integração definitiva com Korun.

---

## 15. KORUN COMO PARTE DO CONTEXTO

Existe uma oportunidade estratégica em avaliação envolvendo a **Korun**, plataforma já desenvolvida por Gabrielle Barros para análise de dados e inteligência para tomada de decisões.

A lógica conceitual discutida foi:

***Korun responde "o que está acontecendo?"***

Enquanto:

***FeelMind responde "o que devemos fazer?"***

E uma camada de **Creative Intelligence** poderia responder:

***"Como isso deve ganhar forma?"***

Essa integração é uma possibilidade estratégica importante, mas **não deve ser tratada como requisito técnico fechado** até que a parceria/integração seja efetivamente definida.

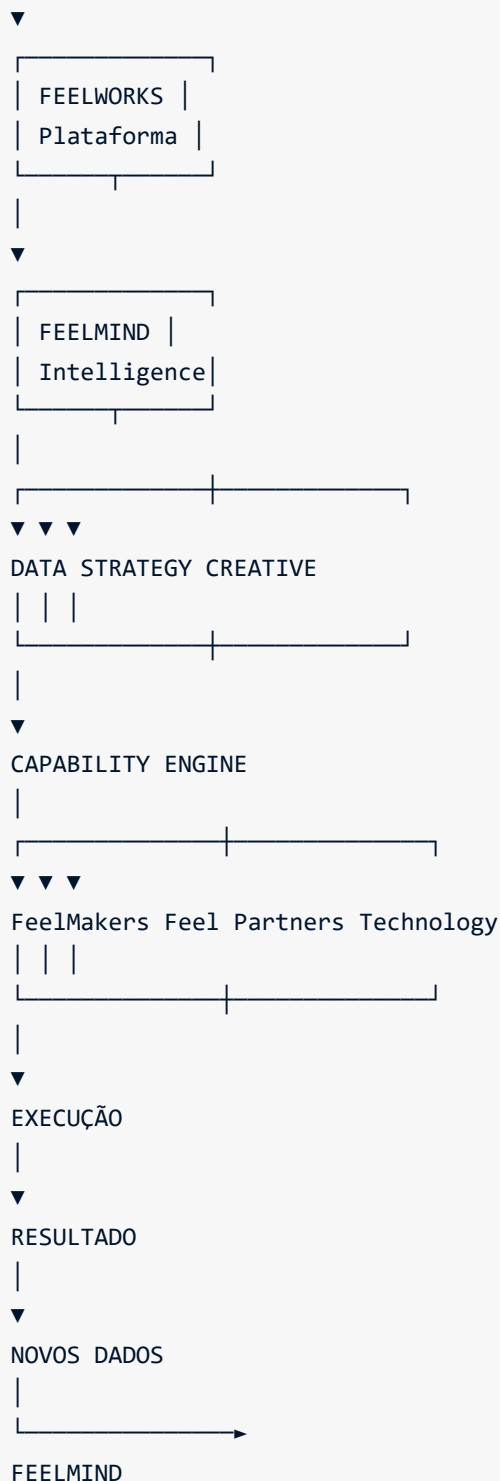
---

## 16. ARQUITETURA CONCEITUAL DE LONGO PRAZO

A visão mais completa pode ser representada assim:

CLIENTE

|



O elemento central é o **feedback loop**:

**Dados → Inteligência → Decisão → Execução → Resultado → Aprendizado → novos dados.**

Esse loop é uma parte fundamental da tese tecnológica do produto.

## 17. RESUMO PARA A IA QUE FARÁ A PESQUISA

Se fosse necessário condensar todo o requisito em uma única especificação:

***Estamos procurando uma plataforma Open Source, preferencialmente self-hosted, capaz de servir como fundação para uma aplicação multi-tenant B2B/B2B2C orientada a workflows,***

***com banco relacional, APIs, autenticação/RBAC, frontend altamente customizável, integrações externas, automações, RAG, LLMs, agentes e human-in-the-loop.***

***O produto será a infraestrutura tecnológica do ecossistema Feel: uma plataforma que conecta clientes, inteligência de dados, IA, projetos, criação, profissionais e empresas parceiras.***

***O MVP deve começar pequeno, potencialmente como uma ferramenta interna/AI-enabled para gestão de dados, métricas, calendário, conteúdo e recomendações, mas a arquitetura precisa permitir evolução para uma plataforma comercial multi-tenant com Brand Memory, Creative Intelligence, Capability Network, matching de profissionais/agências, gestão de projetos, assets, aprovações, reputação, certificações e agentes de IA.***

***A prioridade não é encontrar a ferramenta que permite criar um protótipo mais rapidamente; é encontrar a melhor combinação entre velocidade de desenvolvimento assistido por IA e liberdade arquitetural para evoluir posteriormente para software próprio.***

## Conclusão arquitetural

O ponto mais importante para a próxima etapa de pesquisa é **não procurar "o melhor Lovable open source" genericamente.**

A pergunta correta é:

***Qual plataforma Open Source oferece o melhor ponto de partida para construir uma aplicação AI-native, workflow-driven, multi-tenant e altamente customizável, permitindo começar com Low-Code/AI-Assisted Development e gradualmente migrar para uma arquitetura de software própria sem reescrever o produto inteiro?***

Isso muda bastante o conjunto de ferramentas que deve ser investigado.

E eu recomendaria que a próxima IA avaliasse as candidatas em **três categorias separadas**, porque provavelmente nenhuma ferramenta será perfeita em todas:

1. **AI App Builders / Low-Code** — velocidade de construção.
2. **Backend / Workflow / Agent Platforms** — inteligência e orquestração.
3. **Application Frameworks / Full-Stack Open Source** — controle arquitetural.

O resultado ideal pode inclusive ser uma **combinação de duas ou três tecnologias**, e não uma única plataforma monolítica.